

Debugging

- ▶ Programar es también adquirir habilidades y buenas prácticas, además de poder codificar el problema en un lenguaje de programación específico.
- ▶ En la programación imperativa logramos nuestro objetivo cuando, partiendo de un estado inicial llegamos a un estado final que cumple nuestro propósito. A veces no es simple entender si lo estamos haciendo de forma correcta pues hay muchos estados intermedios.
- ▶ Una habilidad importante para poder comprender esta sucesión de estados es la de poder analizar el código paso a paso.

A esto llamamos *debug*.

First actual case of bug being found

El primer *bug* documentado ocurrió el 9 de septiembre de 1947, cuando un equipo de ingenieros, liderado por Grace Hopper, trabajaba en la computadora Harvard Mark II, un equipo electromecánico de gran tamaño. Durante una de las operaciones, la máquina comenzó a fallar y no funcionaba como se esperaba.

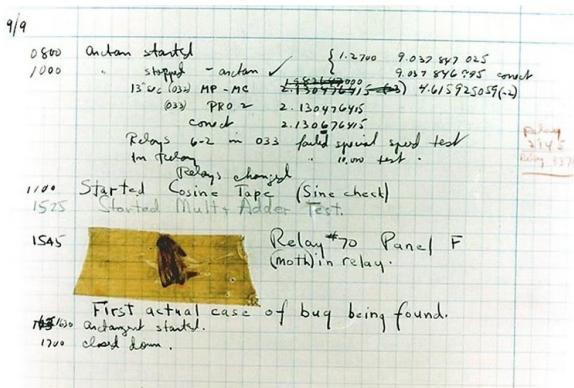


Figura: primer bug documentado en el diario de Grace Hopper 1947

Grace Hopper a cargo del equipo Mark II

Después de investigar, los ingenieros encontraron que la causa del problema era un insecto atrapado entre los relés del sistema. El intruso resultó ser una polilla, que interfería con el flujo de corriente eléctrica.



Figura: Grace Hopper Mark II

¿Qué es Debugging y para qué sirve?

1. Podemos ir paso a paso analizando los valores de las variables durante la ejecución
2. Sirve para poder realizar seguimiento del código
3. Podemos avanzar paso a paso o saltar al siguiente breakpoint
4. Podemos terminar la ejecución por la mitad o bien continuar hasta el final
5. Con VSCode podemos agregar breakpoints durante el momento de debugging, o eliminarlos
6. Se pueden agregar breakpoints con condiciones lógicas, por ejemplo: `valor_actual = 7`

Agregar un breakpoint (punto de detención) en el código

Debemos hacer click a la izquierda del número de línea para agregar el punto de detención en esa línea:

```
1
2  def suma_total(s:[int])-> int:
3      total:int = 0
4      indice_actual:int = 0
5      longitud:int = len(s)
6
7      while (indice_actual < longitud):
8          valor_actual:int = s[indice_actual]
9          total = total + valor_actual
10         indice_actual += 1
11
12     return total
13
```



Figura: Agregamos un breakpoint en la línea 7 del código

Ejecutar con Debug

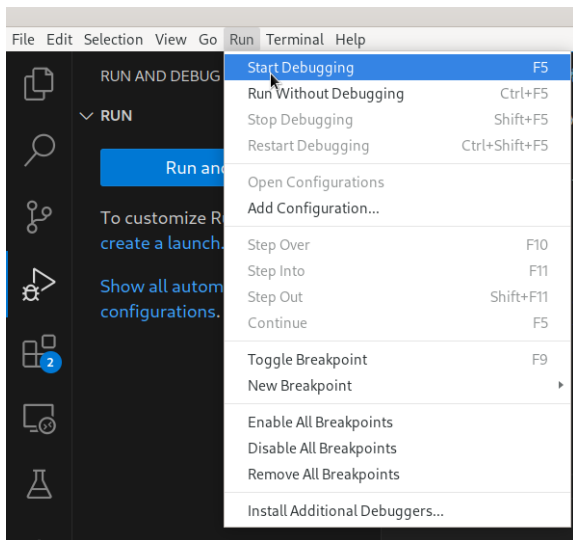
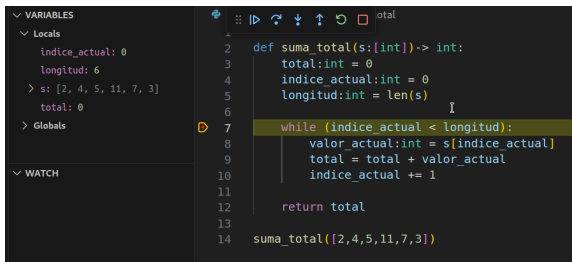


Figura: Ejecutamos el código con la opción Debug

Usamos los controles de la IDE para desplazarnos



The screenshot shows a Python IDE with a debugger window on the left and a code editor on the right. The debugger window has two tabs: 'VARIABLES' and 'WATCH'. The 'VARIABLES' tab is active, showing the 'Locals' section with the following variables and values:

- `indice_actual`: 0
- `longitud`: 6
- `s`: [2, 4, 5, 11, 7, 3]
- `total`: 0

The 'WATCH' tab is empty. The code editor shows a Python function `suma_total` with the following code:

```
1 def suma_total(s:[int])-> int:
2     total:int = 0
3     indice_actual:int = 0
4     longitud:int = len(s)
5
6     while (indice_actual < longitud):
7         valor_actual:int = s[indice_actual]
8         total = total + valor_actual
9         indice_actual += 1
10
11     return total
12
13 suma_total([2,4,5,11,7,3])
```

The code is currently paused at line 7, which is highlighted in yellow. The cursor is positioned at the end of the line. The 'total' variable is visible in the top right corner of the code editor.

Figura: Podemos ver las variables con sus valores al momento del break y usar los controles para movernos

Usamos los controles de la IDE para desplazarnos



F5 Continuar hasta el siguiente breakpoint (o si no hay más hasta el final)

F10 Siguiente paso saltando ingresar a la función que se esté evaluando en esta línea

F11 Siguiente paso ingresando a la función que se esté evaluando en esa línea

Shift+F11 Salir de la evaluación de la función a la que se ingresó

Ctrl + Shift + F5 Reiniciar el debug desde el principio

Shift + F5 Detener el debugging